

Universidad Autónoma de Yucatán

Facultad de Matemáticas

Asignatura:

Programación Orientada a Objetos

Proyecto: UniRide

Hood Code Department:

Moguel Miranda, Nicolas

Uicab Gutierrez, Gadiel Abdias

Profesor:

Lic. Miguel Ángel Canul Chin

Mérida, Yucatán. a 21 de Junio de 2026



1. Introducción y Objetivos

1.1 Contexto y Definición del Problema

El medio de transporte es principalmente uno de los problemas más frecuentes a la hora de desplazarse hacia la universidad. Para ello, el gobierno ha implementado rutas de transporte público para mitigar este problema; esto implica que los estudiantes suelen usar esta facilidad, así ocurre una liberación de la carga de cierto modo.

Sin embargo, la saturación de este medio suele incrementar en horas muy concurridas, debido al porcentaje tan alto de personas que lo usan (no únicamente estudiantes), lo que a su vez ocasiona conglomeraciones, por lo que ciertas rutas van a su capacidad máxima de pasajeros. Como consecuencia, el estudiante tiene la posibilidad de retrasarse. Otro de los problemas comunes relacionado a este es que hay una gran parte de estudiantes que no tienen una ruta directa o viven en fraccionamientos lejanos; por esta razón se tiene que tomar más de una ruta para llegar a su destino y, derivado de esto, se obtiene que el estudiante posiblemente se retrase.

1.2 Justificación del Proyecto

Para resolver estos problemas y eficientar el tiempo/transporte de ciertos estudiantes se desarrollará una solución que consiste en una plataforma de transporte compartido exclusiva para la comunidad universitaria.

1.3 Objetivo General y Alcance

Los estudiantes (conductores y pasajeros) registran sus horarios de clases, desde qué zona salen, hacia qué campus van y cuántos minutos de tolerancia tienen para esperar. El sistema cruza automáticamente todas estas variables y hace el emparejamiento, conectando a quienes tienen rutas y horarios compatibles para que compartan el viaje y los gastos de forma segura.

2. Funcionalidades principales

2.1.1 Sincronización y Emparejamiento por Horarios

La principal Diferencia radica en que no es un simple viaje aislado sino aquí los estudiantes tienen muchas más facilidades, alguna de las funcionalidades pensadas a implementar es la de cargar el horario de su carga académica o bien darle al usuario la facilidad de configurarlo manualmente en caso de errores o simple preferencia.

- **Conductor:** Registra la hora de salida de casa y termino de ultima clase
- **Pasajero:** Registra su hora de entrada y salida
- **Motor lógico:** Compara ambos horarios. Si el pasajero sale a la 1PM y el conductor a la 1:15PM, el sistema detecta una superposición viable para el viaje de regreso

2.1.2 Filtro de Tolerancia por Tiempo de Espera

Dado que no todos los horarios coinciden exactamente el usuario podrá filtrar por tiempo de espera al que está dispuesto esperar, esto amplia la coincidencia con otros usuarios.

- Si un conductor pasa por el fraccionamiento a las 6:30 AM, pero el pasajero prefiere salir a las 6:45 AM y configuró una tolerancia de 20 minutos, el sistema hace Match (porque 6:30 AM entra en el rango de tolerancia de espera).
- Si el pasajero solo está dispuesto a esperar 5 minutos, el viaje se cancela o bien busca coincidencias conforme a lo dado.

2.1.3 Búsqueda de ruta más eficiente

La ruta consiste en un punto de partida y un punto de destino, se utilizará el algoritmo de Dijkstra (el algoritmo para encontrar el camino más corto entre dos nodos) para encontrar la ruta más eficiente y así evitar perjudicar al conductor.



Metodología

El proyecto se desarrolló bajo el paradigma de **Programación Orientada a Objetos** en lenguaje Java, modelando cada entidad del dominio como una clase con una responsabilidad única.

Modelado del dominio

El usuario se representa con la clase abstracta **User**, de la cual heredan **Driver** (conductor) y **Passenger** (pasajero). Cada usuario tiene un origen, un destino y un horario, este último modelado por la clase **Schedule**. La tolerancia de espera se guarda en el propio usuario y la desviación máxima permitida en el conductor.

Representación del mapa como grafo

El mapa se modela como un **grafo ponderado** de aproximadamente 140 nodos. Cada nodo (clase **Node**) representa una ubicación con sus coordenadas, y las aristas representan las conexiones entre ubicaciones. El grafo (clase **Graph**) se construye leyendo archivos de texto mediante clases especializadas:

- **ReadNodes**: lee las coordenadas y crea los nodos.
- **AddAllAdjacents**: genera las adyacencias base entre nodos.
- **ReadAddColumns**: agrega las adyacencias por columnas.
- **ReadRemoveAdjacents**: elimina las adyacencias que no forman parte del mapa.

Cálculo de la ruta más corta

Para calcular la ruta del conductor entre su origen y su destino se utiliza el **algoritmo de Dijkstra** (clase **Dijkstra**), que encuentra el camino más corto sobre el grafo ponderado. Esta ruta es la base para decidir qué pasajeros son compatibles.

Emparejamiento (matchmaking)

La clase **MatchMaker** evalúa a cada pasajero contra el conductor considerando dos criterios:

- **Horario**: compara los horarios con la clase **Schedule** según la tolerancia de espera del

pasajero.

- **Distancia:** calcula la distancia del pasajero a la ruta del conductor y la compara contra la desviación máxima permitida.

El resultado se encapsula en un objeto **MatchResult** con uno de los estados definidos en **MatchStatus**:

Estado	Significado
MATCH	Conductor y pasajero son compatibles
REJECTED_SCHEDULE	Rechazado por incompatibilidad de horario
REJECTED_DISTANCE	Rechazado porque la distancia excede el límite

Persistencia e interfaz gráfica

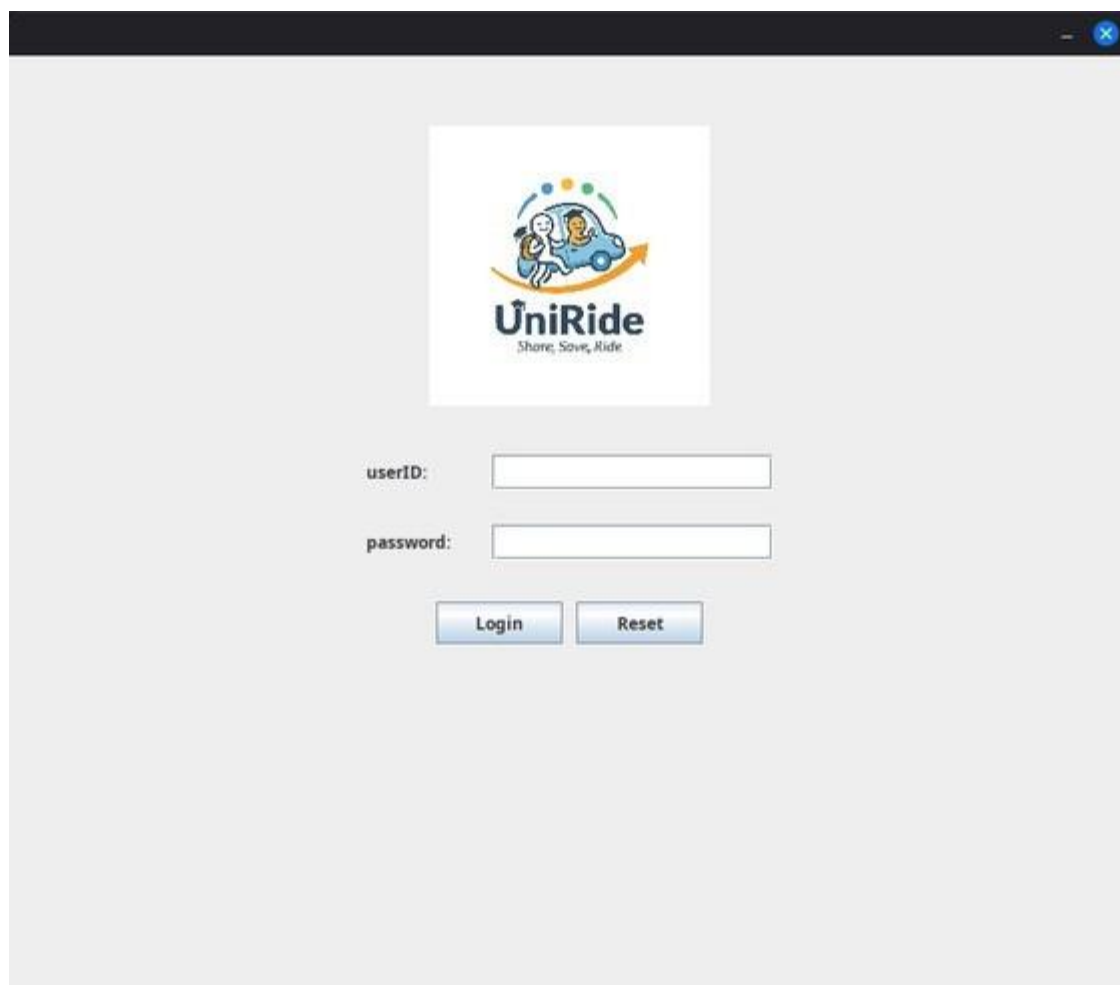
Los usuarios se guardan y se recuperan mediante la clase **DatabaseManager**, que serializa y deserializa la información en un archivo de texto. La interfaz gráfica se construyó con **Java Swing**: **LoginPage** gestiona el inicio de sesión, **Panel** inicializa la ventana principal y **GUI** dibuja el mapa, la ruta del conductor y la tabla de resultados del emparejamiento.

Resultados y Análisis

A continuación se muestran capturas del programa en funcionamiento, ejecutado con los usuarios registrados en el sistema.

Inicio de sesión

Al iniciar el programa se muestra la pantalla de acceso, donde el usuario ingresa su matrícula y contraseña. El sistema valida las credenciales contra los usuarios almacenados y muestra un mensaje según el resultado.



The screenshot shows a web browser window with a dark title bar. The main content area has a light gray background. In the center, there is a white square containing the UniRide logo. The logo consists of a blue car with two people inside, a yellow arrow pointing upwards, and the text 'UniRide' in bold black letters, with 'Share, Save, Ride' in smaller text below it. Below the logo, there are two input fields. The first is labeled 'userID:' and the second is labeled 'password:'. Both fields are empty. Below the input fields are two buttons: 'Login' and 'Reset', both with a blue gradient and white text.

Figura 1. Pantalla de inicio de sesión de UniRide.

Vista del conductor

Al ingresar como conductor se muestra el mapa con su ruta más corta calculada por Dijkstra (en azul), la ubicación de los pasajeros (en verde), el panel con sus datos y la tabla de resultados del emparejamiento.

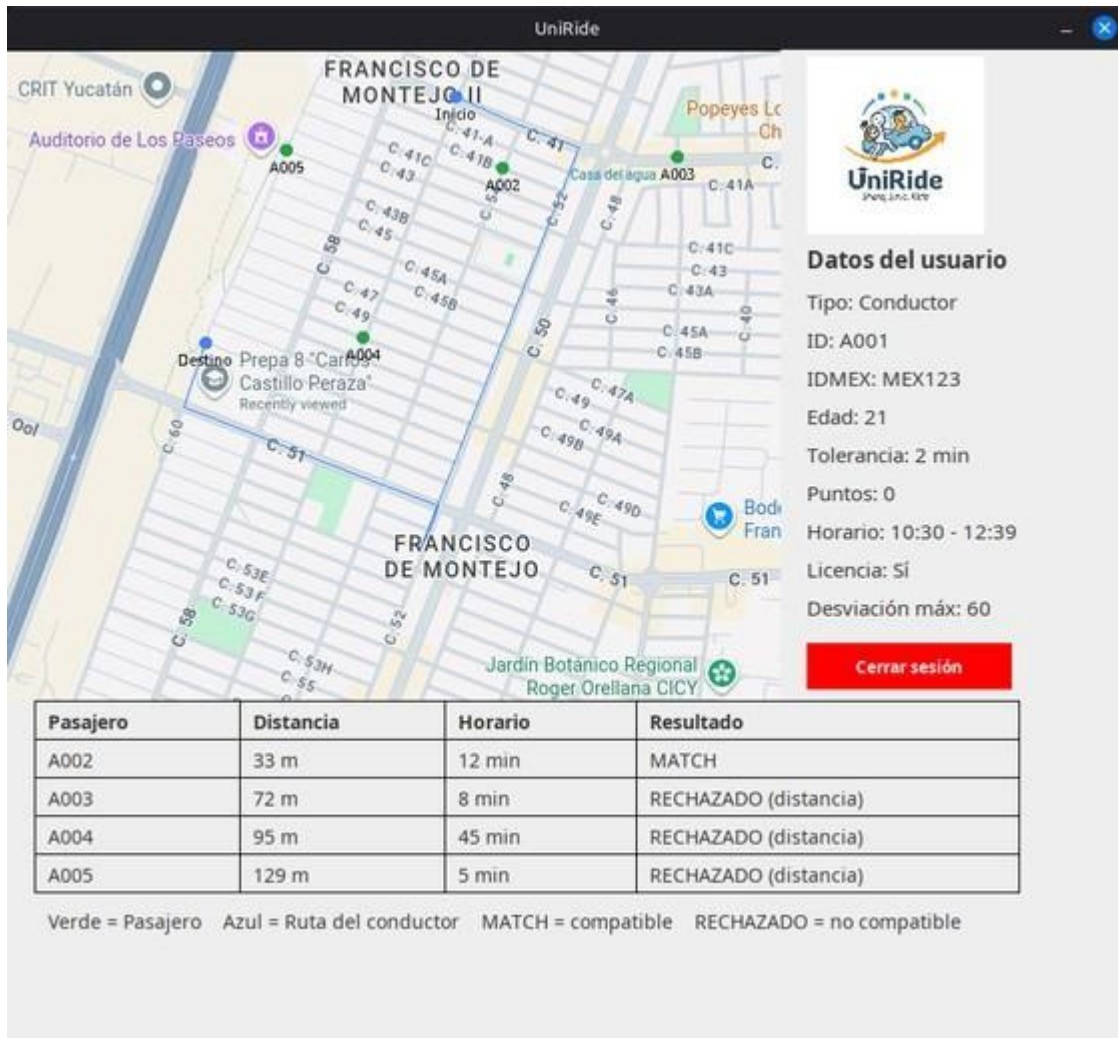


Figura 2. Vista del conductor: ruta, pasajeros y tabla de emparejamiento.

En este ejemplo, el conductor A001 obtiene los siguientes resultados: el pasajero **A002** hace **MATCH** por estar a 33 m de la ruta, mientras que los pasajeros **A003**, **A004** y **A005** son rechazados por distancia (72 m, 95 m y 129 m respectivamente), al superar la desviación permitida. La tabla muestra por cada pasajero su distancia a la ruta, la diferencia de horario y el resultado final, lo que confirma que los criterios de emparejamiento funcionan correctamente.

Vista del pasajero

Al ingresar como pasajero se muestra el mapa con su origen y destino y el panel con sus datos personales, sin la tabla de emparejamiento, que es exclusiva del conductor.

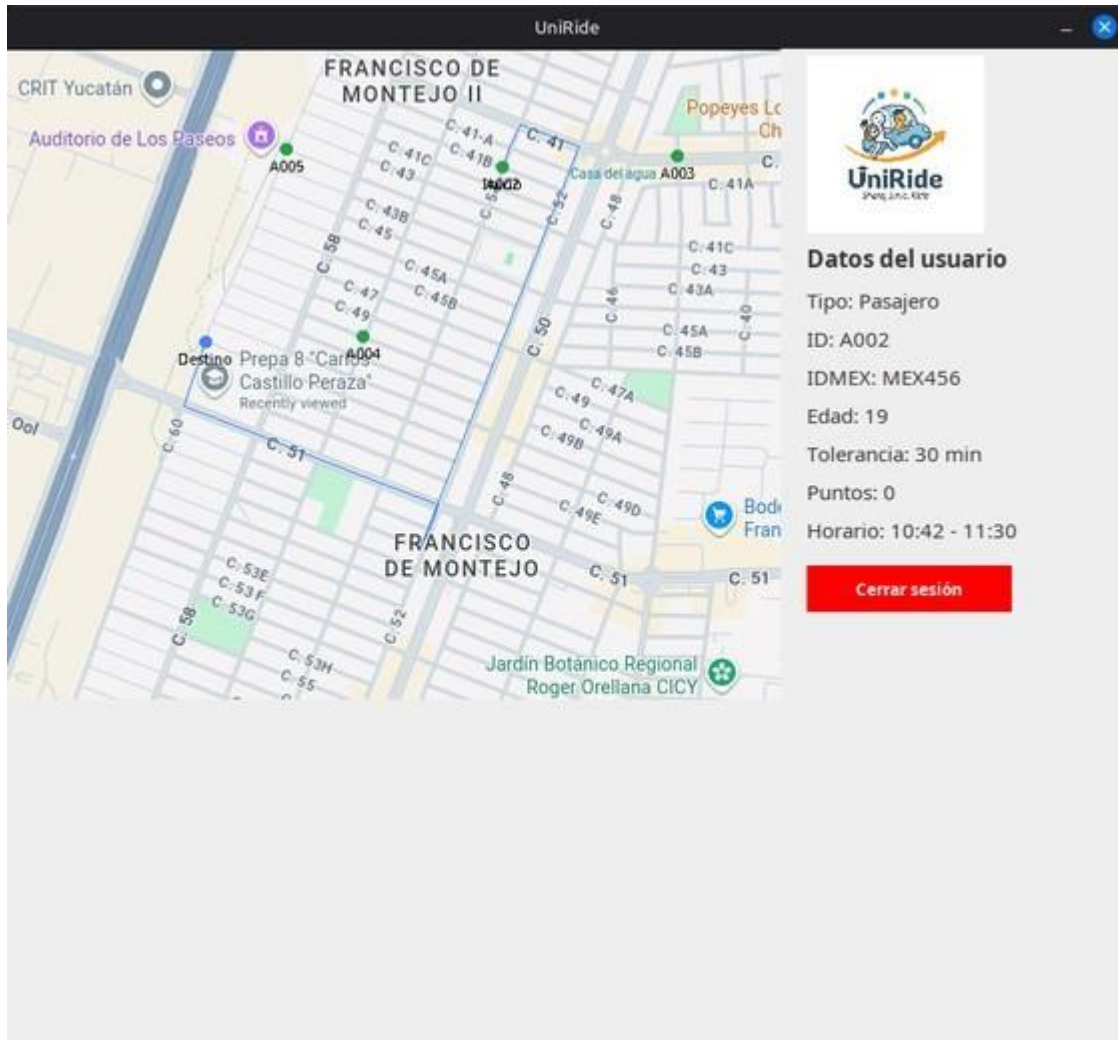


Figura 3. Vista del pasajero con sus datos y ubicación.

Conclusiones

Se cumplió el objetivo del proyecto: desarrollar una plataforma de transporte compartido para la comunidad universitaria que conecta a conductores y pasajeros con rutas y horarios compatibles. El sistema construye el mapa como un grafo, calcula la ruta más corta del conductor con el algoritmo de Dijkstra y realiza el emparejamiento evaluando horario y distancia, mostrando los resultados de forma clara en la interfaz gráfica.

El desarrollo permitió aplicar de forma práctica conceptos de **estructuras de datos** (grafos), **algoritmos** (Dijkstra) y **Programación Orientada a Objetos**, apoyándose en una arquitectura modular de clases con responsabilidades únicas y en los principios de diseño SOLID. Esto facilita el mantenimiento y la extensión del sistema.

Como trabajo a futuro, el proyecto puede evolucionar migrando la persistencia a una base de datos, permitiendo el registro de nuevos usuarios desde la aplicación y ampliando el emparejamiento a varios conductores de forma simultánea.